InfoBridge: Sprint & Hack Roadmap

Team: The North Remembers · **Project:** DSI 9A (Hyderabad Runners)
Covers: Sprint 2 (27 Sep) → Showcase (20 Nov 2026)

Overview

Sprint 2 lays the foundations: project structure, synthetic dataset and a baseline. **Hack 1 builds the core of InfoBridge, about 60% of the project**, and ends with a working demo video. Later sprints extend, test and polish it. Every phase is measured on the same synthetic test cases, so progress shows up as numbers.

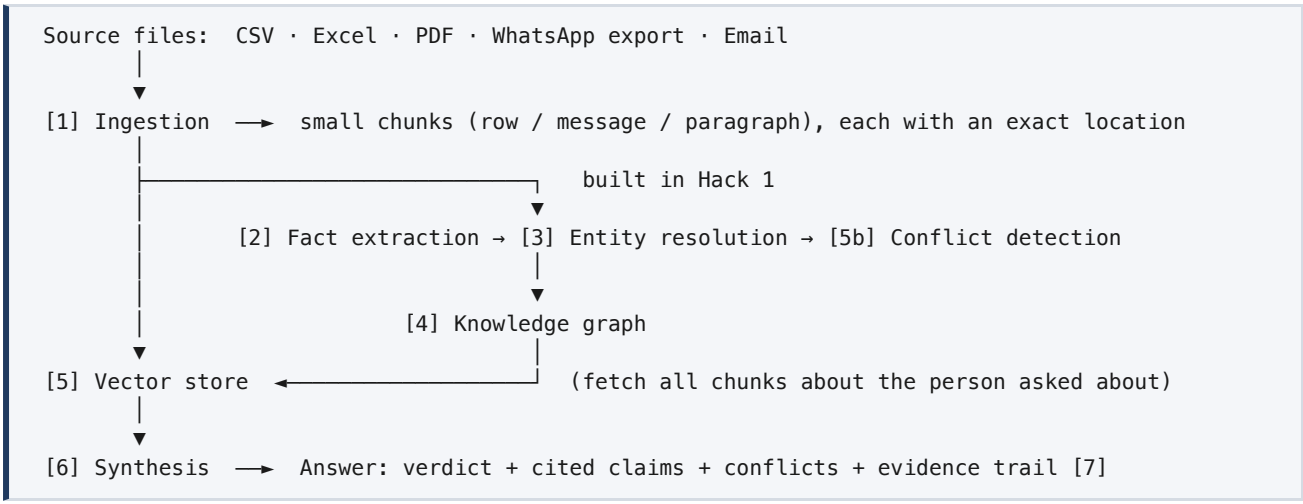
Phase	Dates	Version	Focus	Project done	Key output
Sprint 2	27 Sep – 10 Oct	v0.1 foundations	Boilerplate, synthetic dataset, baseline	~30%	Technical Design Doc, repo, dataset
Hack 1	10 Oct	v0.1	**Full core pipeline working end to end**	**~60%**	Working demo video, revised doc
Sprint 3	11 – 24 Oct	v0.2	Act on feedback, more cases, open data, better matching	~75%	Test spec and results
Sprint 4	25 Oct – 7 Nov	v0.9	Systematic testing, edge cases, robustness	~90%	Evaluation and failure analysis
Hack 2	7 Nov	v1.0	Polish and final evaluation	100%	Final Technical Design Doc
Sprint 5	8 – 14 Nov	v1.0 final	Package and defend	–	Slides, poster, video, final repo

---

Sprint 2: Foundations (boilerplate + dataset + baseline)

Goal: everything Hack 1 needs to build on: a project skeleton with fixed component interfaces, a synthetic dataset with answer keys, and a baseline score.

Architecture



What's ready after Sprint 2

Part	Status	Notes
Project skeleton	Done	One Python module per component, with fixed inputs/outputs (`models.py`)
Synthetic dataset	Done	3 cases (simple / medium / conflict), 9 source files in 5 formats, answer keys (`gold.json`)
[1] Ingestion	Done	CSV, Excel, PDF, WhatsApp, email → chunks with exact locations
[5] Vector store	Done	ChromaDB + all-MiniLM-L6-v2
[6] Synthesis (baseline)	Done	Gemini answers with citations; citation check; "not enough evidence" fallback
[7] Evidence trail (basic)	Done	Source → chunk → claim
Evaluation script	Done	Retrieval recall, fact recall, citation validity, conflict recall, verdict, latency
[2] [3] [4] [5b]	Interfaces only	Built in Hack 1

Test cases

Case	Sources	Question	What makes it hard
1 – Simple	Events sheet, email	Details of the 25 Oct long run	Nothing. This is the control case
2 – Medium	Volunteer roster, WhatsApp, first-aid PDF	Who's at Aid Station 3, and are they trained?	Same people with different names (*Srini = Srinivas Rao Kumar*); a look-alike (*Srinivas Reddy*)
3 – Conflict	Members sheet, registrations, WhatsApp, rules PDF	Is Ravi Sharma eligible for the training programme?	*Ravi Sharma / R. Sharma / Ravi S.*; look-alike *Ravi Shankar*; 2 contradictions between sources

Baseline (retrieval only)

Case	Retrieval recall	Fact recall	Citation validity	Conflict recall
1 – Simple	1.00	1.00	1.00	–
2 – Medium	0.67	0.50	1.00	–
3 – Conflict	0.57	0.14	1.00	0.00

When names differ across sources, the baseline can't find the right records. Ravi Sharma's own membership row ranks 20th of 29, below the look-alike Ravi Shankar, and conflicts are never flagged. **Hack 1 fixes exactly this.**

Hack 1 (10 Oct): Build the core (→ ~60%)

Goal: the full pipeline from the Design Doc running end to end on all 3 cases, shown in a **working demo video**.

Demo target: ask "*Is Ravi Sharma eligible for HMTP Batch 2026B?*" and get:

- verdict **YES**
- every eligibility rule backed by a citation to the exact row or message
- **both conflicts flagged**, each with both values and their sources

- *Ravi Shankar* never mixed in

What gets built

#	Component	What it does	Done when
A	**Fact extraction**	Spreadsheet columns → facts directly; Gemini extracts facts from chats, PDFs and emails; dates and phone numbers normalised	Facts extracted for all 3 cases
B	**Entity resolution**	Same phone/email → same person; compatible names (*R. Sharma* ~ *Ravi Sharma*, *Srini* ~ *Srinivas*); fuzzy matching; never merge different surnames	Entity resolution F1 ≥ 0.85; look-alikes kept apart
C	**Conflict detection**	Same person + same attribute + different values → conflict, with severity and a suggestion	Both planted conflicts found; nothing flagged in cases 1–2
D	**Knowledge graph + retrieval**	NetworkX graph of people, events and facts, each linked to its source; retrieval fetches every chunk about the person asked about	Case 3 retrieval recall ≥ 0.90
D	**Conflict-aware answer**	Detected conflicts go into the prompt and the answer	Answer shows conflicts with sources
E	**Demo app**	One-page Streamlit app: question box → verdict, cited claims, conflict cards, evidence trail. A demo harness, not a product frontend	Runs reliably for the video
E	**Evaluation + video**	Re-run the evaluation; before/after table; record the demo	Video and doc ready

Prep today (9 Oct), so the hack is spent building

- Assign owners A–E (with 4 people, one person takes both D and E).
- Everyone: clone the repo, run `pytest`, get their own Gemini key (separate keys avoid rate limits), install the Hack 1 libraries.
- Agree on the list of attribute names (e.g. `medical_clearance_date`, `registered_on`, `batch_start_date`).
- B and C: write tests using hand-made facts from case 3, so they don't wait for A.
- E: write the demo script and set up screen recording.

Hack-day timeline (about 5 h)

Time	What happens
0:00 – 0:15	Pull, run tests, recap the plan
0:15 – 2:00	Build A–E in parallel (B and C start from fake facts)
2:00 – 2:15	**Sync:** A's real facts go to B and C
2:15 – 3:30	Connect everything in the pipeline; demo app wired to the real pipeline
3:30 – 4:00	Run the evaluation; fix the worst failure
4:00 – 4:30	Record the demo video (plus a backup take)
4:30 – 5:00	Update the doc: results, what works and what doesn't, questions for instructors

If we fall behind: cut LLM extraction from chats down to a simple date finder (a date near words like *clearance* or *begins*), which still catches both conflicts. Never cut the demo video.

Demo video (about 3 min)

1. **Problem (20 s):** 4 messy files about one runner: three name forms, a look-alike, two contradictions.

2. **Before (30 s):** the baseline misses his records and doesn't mention the conflicts.
3. **After (60 s):** the same question in the demo app gives YES, cited claims, and 2 conflict cards.
4. **Evidence (30 s):** click a citation and trace it back to `members.csv` row 5.
5. **Numbers (20 s):** before/after metrics table.
6. **What doesn't work yet, and questions for instructors (20 s).**

Deliverables: revised Technical Design Doc, demo video, PoC v0.1 on GitHub, feedback log for Sprint 3.

Sprint 3 (11 – 24 Oct): Improve & extend (→ ~75%)

- Act on Hack 1 feedback; fix the biggest failures first.
- **More test cases:** 6+ in total, with new question types (volunteer lists, event summaries, incident follow-ups). At least one written by a teammate who didn't build the matcher.
- **One real open-data scenario** (OGD India / WHO / OpenAlex) to show the approach works beyond our own data.
- **Better entity matching:** phonetic matching, ambiguous names (*Ravi S.*), confidence scores.
- **Better scoring:** an LLM-based or manual check of answers, in addition to keyword matching.

Deliverables: PoC v0.2, test spec and results, known limitations.

Sprint 4 (25 Oct – 7 Nov): Validate & harden (→ ~90%)

- **Systematic testing:** unit tests per component plus end-to-end tests on all cases.
- **Edge cases:** empty files, questions with no answer in the sources, heavily conflicting sources, near-identical names.
- **Robustness:** handle LLM errors and rate limits; cache results; fail with a clear message rather than silently.
- **Performance:** measure and reduce latency.
- **Failure analysis:** what still goes wrong, and why.

Deliverables: revised Technical Design Doc (evaluation, failure analysis, limitations), PoC v0.9, demo plan for Hack 2.

Hack 2 (7 Nov): Complete & polish (→ 100%)

- Fix the remaining high-priority issues; make outputs clean and the demo reliable.
- Final evaluation run; a results table comparing every version (v0.1 → v1.0).
- Prepare a backup recording in case the live demo fails.

Deliverables: final Technical Design Doc, PoC v1.0 on GitHub.

Sprint 5 (8 – 14 Nov): Package & defend

- Clean the code; write the README (setup, how to run, architecture).
- Final versions of the Functional Scope Doc, Design Doc and Technical Design Doc.
- **Slides, poster and demo video:** problem → solution → technology → results.

- Review with NGOs (table discussions).
- Prepare for likely questions:
  - "How is this different from ChatGPT?"
  - "Would it work on real NGO data?"
  - "What would production need?"
  - "Which other NGOs could use it?"

Then: Demo 2 (14 Nov) · Final assessment (18 Nov) · Showcase (20 Nov).

How the metrics should improve

Metric	v0.1 baseline	Hack 1	v0.2	v1.0
Retrieval recall (case 3)	0.57	≥ 0.90	≥ 0.90	≥ 0.95
Conflict recall	0.00	1.00 (both planted conflicts)	≥ 0.90 on all cases	≥ 0.90
False conflicts (cases 1–2)	–	0	–	–
Entity resolution F1	–	≥ 0.85	≥ 0.90	≥ 0.90
Citation validity	1.00	1.00	1.00	1.00
Verdict accuracy (case 3)	TBD	1.00	≥ 0.90 on all cases	≥ 0.90

These are targets on our synthetic cases only. They don't show that the system would work on real NGO data; understanding that gap is part of the project.